IIT HYDERABAD

CS5903 – Distributed AI Training

AdaptEngine

3. Adaptive Learning, Diagnostic Assessment and Personalized Path-Planning Engine

Assignment 2

Vuppula Hemanth Reddy

AI23BTECH11033

Group 15

Repository: https://github.com/vhemanth06/AdaptEngine

Contents

1 Introduction **3**

1.1 Motivation . . . 3

1.2 Project Milestones . . . 3

2 Proposed Approach **4**

2.1 Proposed Solution . . . 4

2.2 Methodology . . . 4

2.2.1 Curriculum Representation (M1) . . . 4

2.2.2 Mastery Model . . . 5

2.2.3 Evidence State and Uncertainty . . . 5

2.2.4 Adaptive Diagnostic (M2) . . . 6

2.2.5 Difficulty Policy (M4) . . . 6

2.2.6 Time-Budgeted Planner (M3) . . . 7

2.2.7 Spaced Review and Remediation (M5) . . . 7

2.2.8 Dynamic Replanning (M6) . . . 8

2.2.9 Decision Traces and Explanations (M7) . . . 8

2.2.10 Comparative Evaluation (M8) . . . 8

2.2.11 Optional Extension . . . 8

2.3 Why This Approach Is Suitable . . . 9

2.4 Relation to Existing Work . . . 9

3 System Design **10**

3.1 Design Principles . . . 10

3.2 System Architecture . . . 11

3.2.1 Core Layer . . . 11

3.2.2 Domain Layer . . . 11

3.2.3 Decision Layer . . . 12

3.2.4 Orchestration Layer . . . 12

3.2.5 Interfaces Layer . . . 12

3.2.6 Optional Agentic Layer . . . 12

3.3 System Context . . . 13

3.4 Major Components . . . 13

3.5 Component Interaction and Workflow . . . 14

3.6 Inputs and Outputs . . . 15

3.7 Data Model . . . 16

4 Technical Feasibility **16**

4.1 Tools, Frameworks, Datasets and Infrastructure 16

4.2 Technical Feasibility of the Design 17

4.3 Evaluation Protocol 18

4.4 Potential Implementation Challenges, Assumptions and Limitations 19

5 Implementation Plan **20**

5.1 Current Status 20

5.2 Task Breakdown 21

5.3 Core functionalities to be implemented by the end 22

5.4 Acceptance Tests and Traceability 23

5.5 Expected Outcomes of the Implementation Phase 25

5.6 Planned Next Steps 25

6 Conclusion **25**

1 Introduction

Online courses usually present lessons in one fixed order for every learner. In reality, learners differ a lot. Some have never seen the topic, some already know half of it, and their deadlines and free time also vary. A fixed sequence cannot suit both a beginner who has only 10 minutes a day and an experienced person who wants a short refresher. The first learner gets overwhelmed by lessons that are too hard or too long, while the second wastes time on things they already know.

1.1 Motivation

The motivation behind AdaptEngine is to replace this one-size-fits-all sequence with a personalization engine. Such an engine should find out what a learner already knows, plan what to study within their goal, deadline and daily time limit, and keep adjusting as they progress. It must do this while always respecting the order in which topics depend on each other. It should also explain why each activity was chosen, so that the learner can trust the plan.

Building this is difficult for a few reasons:

- The initial skill level has to be estimated from a very short diagnostic, and the estimate must carry uncertainty instead of being a single number.
- Prerequisites are strict rules, whereas time, deadline and difficulty are things to optimize. If a goal cannot fit in the available time, the system must say so clearly and not silently break a rule.
- New lessons, remediation and spaced review all compete for the same few minutes each day, so a clear priority policy is needed.
- After missed work or changed constraints, the new plan must remain valid and keep everything already completed.
- Every inclusion, omission, review or difficulty change must be traceable to learner state, graph constraints or a policy score.

1.2 Project Milestones

AdaptEngine is a backend policy and recommendation engine for any subject whose content can be organised as a prerequisite graph of concepts and whose exercises can be graded as correct or incorrect. The prototype is demonstrated on a small Python curriculum, but the engine itself does not depend on the subject. It is used through a Python API and a thin command-line interface. For a given learner, the complete system is designed to:

- **Represent the curriculum** as a prerequisite graph of concepts, lessons and exercises with durations (M1).
- **Estimate mastery** of each concept, together with its uncertainty, using a short bounded adaptive diagnostic (M2).
- **Generate a day-by-day study plan** that never breaks prerequisites and fits the learner's goal, deadline and daily budget of 5, 10, 15 or 30 minutes, or reports clearly that the goal is infeasible (M3).

- **Update mastery** after every quiz or exercise and **adapt difficulty** using an explicit rule (M4).
- **Schedule spaced review and remediation** based on forgetting risk, prerequisite importance and remaining daily time (M5).
- **Replan** when the learner misses a day, fails an assessment, or changes the deadline or daily budget, while keeping completed work (M6).
- **Explain every decision** through machine-readable decision traces (M7).
- **Evaluate it** against fixed, static and greedy baselines on diagnostic accuracy, plan feasibility, runtime, prediction and calibration, and simulated learning gains (M8).

The full learning platform, real course authoring and private learner data are outside the scope of this project. The aim is a working, documented prototype and an evaluation report that shows different valid plans for synthetic learners with different skills, deadlines and time budgets, together with a measurable improvement over non-adaptive baselines.

2 Proposed Approach

2.1 Proposed Solution

AdaptEngine is designed as a deterministic engine in which a mastery model estimates what the learner knows and a set of hard constraints decides what is allowed. The engine answers three questions for every learner: *what does this learner already know*, *what should they do next within their time*, and *why*. The overall pipeline is

$$\begin{aligned}&\text{validated curriculum} \to \text{diagnose} \to \text{eligible activities} \to \text{deterministic ranking}\\ &\to \text{budgeted schedule} \to \text{events} \to \text{replan}.\end{aligned}$$

The central design rule is the separation between **validity** and **preference**:

- **Hard constraints** (prerequisites and the daily time budget) are checked by the engine and can never be traded away.
- **Soft objectives** (review urgency, path unlocking, difficulty fit) are scores used only to *rank* activities that have already passed the hard filter.

A ranking score is therefore never responsible for checking whether a rule is violated. If a goal cannot be met, the engine reports it as infeasible and does not silently break a rule.

2.2 Methodology

2.2.1 Curriculum Representation (M1)

The curriculum is a directed acyclic graph (DAG) in which nodes are concepts and edges are mandatory prerequisites. Each concept has lessons and exercises, and each activity has a target concept, an indivisible integer duration in minutes, and (for exercises) a difficulty $d \in [0.05, 0.95]$. The graph is handled by NetworkX for topological order, ancestor queries and cycle detection.

Content is trusted only after passing a gate: schema validation, rejection of unknown prerequisite references and cycles, and a review check against a SHA-256 manifest. This gate is applied at every live entry point so that altered or unreviewed content cannot reach the decision logic. The pipeline is

$$\text{draft content} \rightarrow \text{human review} \rightarrow \text{hash recorded} \rightarrow \text{live curriculum}.$$

2.2.2 Mastery Model

A single BKT-style model is used for every concept, so that one consistent state representation drives diagnosis, planning and review. Let p be the current probability that the learner has mastered a concept. For an exercise of difficulty d, the guess and slip probabilities are

$$g(d) = 0.30\,(1-d), \qquad s(d) = 0.05 + 0.25\,d,$$

so easy items are more likely to be answered correctly without mastery, and hard items are more likely to be missed even with mastery. The predicted probability of a correct answer is

$$P_c(p,d) = p\,(1-s) + (1-p)\,g.$$

This quantity is used for next-item selection and for calibration (Brier score). After a response, the posterior is

$$p'_{\text{correct}} = \frac{p(1-s)}{p(1-s)+(1-p)g}, \qquad p'_{\text{wrong}} = \frac{ps}{ps+(1-p)(1-g)}.$$

A learning transition then models the effect of practice:

$$p'' = p' + (1-p')\,T, \qquad T_{\text{diag}} = 0,\ T_{\text{exercise}} = 0.05,\ T_{\text{lesson}} = 0.20.$$

Worked reference. For $p = 0.5$ and $d = 0.5$ we get $g = 0.15$ and $s = 0.175$. A wrong answer gives a Bayes posterior of $p' \approx 0.1707$; applying the exercise learning transition ($T = 0.05$) gives $p'' \approx 0.2122$. The engine stores the final value p'' in learner state. Tests check both the posterior and the final value, so the Bayes formula and the full update are each verified separately.

Parameter status. The values $p_0 = 0.20$, $\theta = 0.85$, T, $g(d)$ and $s(d)$ are *synthetic heuristics*, not fitted parameters, and all of them are kept in one immutable configuration.Here $\theta = 0.85$ is the *mastery threshold*: a concept is considered mastered only when its mastery probability reaches θ (and enough observed responses exist).

2.2.3 Evidence State and Uncertainty

A lesson is instruction, not a measurement, so completing a lesson must not count as assessed mastery. Each learner–concept pair therefore keeps three separate pieces of information:

(i) **Exposure:** lessons completed. They may raise p through T_{lesson} but never mark the concept as assessed.

(ii) **Observed evidence:** the number and outcomes of exercise and diagnostic responses, each with a unique `attempt_id`.

(iii) **Evidence-mastered:** true only when $p \geq \theta$ *and* at least k observed responses exist (proposed $k = 2$).

Prerequisite gating uses *evidence-mastered* only. A replayed `attempt_id` is ignored, so retries cannot inflate mastery, and an event dated earlier than the last practice day cannot rewind it.

Uncertainty is measured using the entropy of the mastery probability,

$$H(p) = -p \log_2 p - (1-p) \log_2 (1-p),$$

where p is the probability that a student has mastered a skill. For example, $H(0.5) = 1.0$, $H(0.2) \approx 0.722$, and $H(0.9) \approx 0.469$. Higher entropy indicates greater uncertainty about whether the student has mastered the skill. However, entropy measures uncertainty about the student's binary mastery state, not the accuracy of the estimated model parameters. It also does not necessarily decrease as more responses are collected. For example, when $p = 0.9$, $d = 0.5$, and the student answers incorrectly, the mastery probability decreases to 0.6495, increasing entropy from 0.469 to approximately 0.93. This means that an unexpected response can increase uncertainty. Therefore, entropy is used only to select items and help determine when to stop, subject to a fixed maximum number of items. The accuracy of the mastery estimates is evaluated separately on simulated student attempts that were not used for model development, using Mean Absolute Error (MAE) and the Brier score.

2.2.4 Adaptive Diagnostic (M2)

The diagnostic must give a calibrated starting estimate from very few questions. The proposed rules are:

- **Cold start:** every concept begins at $p_0 = 0.20$ and is marked `UNASSESSED`, which is treated as unmastered for diagnostic purposes. Existing learner state is preserved and never overwritten.
- **Concept order:** concepts are visited in stable topological order. If a concept ends with $p < 0.40$, its descendants are not probed and remain `UNASSESSED`, since their prerequisite is not met.
- **Item selection:** among unasked exercises of the current concept, choose the one with the largest expected entropy reduction

  $$\mathrm{EIG}(a) = H(p) - \Big[P_c \, H\big(p'_{\text{correct}}\big) + (1 - P_c) \, H\big(p'_{\text{wrong}}\big) \Big],$$

  computed without the learning transition. Ties are broken by smaller $|d - 0.5|$ and then by lexicographic `activity_id`, which keeps selection deterministic.
- **Stopping:** stop when the item cap $N_{\max}$ is reached (proposed 8 items overall and at most 3 per concept), or when every concept in scope has $H(p) \leq 0.50$, that is $p \leq 0.11$ or $p \geq 0.89$.
- **Baselines:** fixed-order and random item selection at the same item budget, used for comparison.

2.2.5 Difficulty Policy (M4)

The next exercise is not chosen by setting difficulty equal to mastery, because under this response model difficulty and mastery are not on the same scale. Instead the policy uses the response model itself: choose the exercise whose predicted success probability $P_c(p, d)$ is closest to a target value of 0.75, so the learner is challenged but not overwhelmed. Ties follow the same

chain as in the diagnostic. After each response the mastery estimate is updated as above. Two consecutive wrong answers on a concept with $p < 0.40$ trigger remediation, and two consecutive correct answers with $p \geq \theta$ mark the concept as evidence-mastered.

2.2.6 Time-Budgeted Planner (M3)

The planner is a deterministic scheduler with:

1. **Hard filter.** An activity is *eligible* only if all its prerequisite concepts are evidence-mastered and its duration fits in the remaining time of the day. Durations are indivisible.
2. **Ranking.** Eligible activities receive a score

$$\text{score} = w_r \cdot \text{review urgency} + w_u \cdot \text{unlock count} + w_d \cdot \text{difficulty fit},$$

   where the unlock count is the number of downstream concepts that depend on the activity's concept. Ties are broken by `activity_id`.
3. **Packing.** Each day is filled in ranked order without ever exceeding the daily budget $B \in \{5, 10, 15, 30\}$ minutes.
4. **Feasibility.** Cheap bounds are checked first: the critical-path time of unmastered concepts against the time remaining before the deadline, and any mandatory lesson longer than the daily budget.A small exact oracle then verifies small instances.

The oracle gives a three-valued answer, `FEASIBLE`, `INFEASIBLE` or `UNKNOWN`. For example, three activities of $8+8+4$ minutes under a 10-minute budget pass the simple time bounds but cannot fit in two days, so the oracle reports `INFEASIBLE` because the workload needs three days. If the search exceeds its time limit it returns `UNKNOWN` and never a false `INFEASIBLE`. An infeasible result always carries the shortfall and corrective options, such as a longer deadline or a larger budget.

2.2.7 Spaced Review and Remediation (M5)

New lessons, remediation and review compete for the same few minutes each day, so a clear priority policy is needed.

- **Forgetting model:** We use a simple model to estimate how much a student may forget over time, $r(t) = 2^{-\Delta t/h}$, where Δt is the number of days since the last practice and h is the number of days required for the retention value to reduce by half. Each concept has its own half-life, stored in the learner's state. This serves as the baseline model. Duolingo's Half-Life Regression may be considered if sufficient training data and suitable input features are available, since features used for language learning may not work directly for other concepts.
- **Review urgency:** a concept becomes a review candidate when its predicted recall r falls below a threshold, and its urgency is weighted by its prerequisite importance (the unlock count).
- **Priority order under one budget:** remediation of failed prerequisites first, then high-urgency reviews, then new lessons.
- **Remediation:** when a failed exercise leaves $p < 0.40$, a remediation lesson is inserted, and after the specified remediation cycles the review state and remediation flag are updated.

2.2.8 Dynamic Replanning (M6)

Four events trigger replanning: a `MISSED` day (treated as exactly zero minutes of activity), a `FAILED_ASSESSMENT`, a `DEADLINE_CHANGED` and a `BUDGET_CHANGED`. Every event is a typed object processed through one boundary, `Engine.record(state, log, event)`, which updates learner state and the execution log together. Replanning recomputes only the remaining horizon, keeps every completed attempt without losing or duplicating it, and returns either a valid revised plan or a proof that the new goal is infeasible.

2.2.9 Decision Traces and Explanations (M7)

Every inclusion, omission, review and difficulty change writes a machine-readable record containing the learner state at decision time, the constraint that applied, the score components, and a reason code. For example, an omitted activity carries a code such as "prerequisite not evidence-mastered" or "already mastered above θ", and a selected activity carries its score breakdown. This keeps every decision traceable to state, graph constraints or a policy score, and lets the explanation coverage metric be computed directly.

2.2.10 Comparative Evaluation (M8)

A transparent simulator is used to generate student responses and track their true mastery levels. Its learning behavior differs from the BKT-style model used by the planner, and at least one additional setting is tested to examine performance under model mismatch. Simulator parameters are selected from ranges reported in public educational datasets, and the source of each parameter is documented.

The proposed method is expected to be compared with five fixed baselines: fixed-order and random diagnostic selection with the same item budget, a static plan based on prerequisite order, a greedy planner without review scoring, and full recomputation during replanning. All methods will be evaluated using the same learner profiles, goals, deadlines, time budgets, and random seeds.

Performance is measured across four areas:
(1) **Constraint adherence**, including prerequisite violations, budget violations, and the proportion of feasible plans;
(2) **Prediction accuracy**, including diagnostic MAE at the same item count, the Brier score for mastery probability P_c, and AUC for predicting correctness on the next attempt;
(3) **Learning outcomes**, including goal mastery by the deadline and retention after review;
(4) **Efficiency and explainability**, including replanning runtime and explanation coverage.

2.2.11 Optional Extension

Learning-based and agentic components are excluded from the core engine. As an optional extension, at most one feedback-driven policy, for example a contextual bandit, may be added to re-rank activities that the engine has already declared eligible. The policy operates strictly downstream of the hard-constraint filter, so it cannot introduce new activities or violate prerequisite and daily-budget constraints.

If implemented, the policy requires a formal specification: the observations (learner state and candidate features), the action set (the eligible activities), the logged selection probabilities, the reward signal, the update rule, and a fallback to the deterministic ranking. Evaluation

compares three variants on identical, paired workloads: a frozen policy, an updating policy, and a random policy. Using logged selection probabilities allows off-policy estimates to be checked against the simulator's outcomes.

The extension is attempted only after the core acceptance tests pass. The deterministic engine retains authority over prerequisites, budgets and learner state at all times.

2.3 Why This Approach Is Suitable

- **Safety by construction.** Prerequisites and budgets are enforced in the eligibility filter and not in a score, so prerequisite and budget violations are prevented by design and checked by tests.
- **Interpretability.** BKT-style posteriors, entropy and explicit score components can be written directly into a decision trace. A neural model cannot supply such triggers.
- **Appropriate for sparse evidence.** A learner gives only a handful of binary responses, which suits a small Bayesian model better than a data-hungry sequence model.
- **Determinism and reproducibility.** Fixed tie-breaks, a hashed configuration and seeded runs give identical decisions for identical inputs.
- **Measurable.** Each module has a frozen baseline and a binary acceptance test, so improvement can be shown and not only claimed.

2.4 Relation to Existing Work

Table 1 summarizes the existing work used in this project and explains which approaches are adopted, modified, or excluded.

Table 1: Existing work and its role in this project.

Area	Existing work	Use in this project
Mastery tracing	Bayesian Knowledge Tracing; pyBKT (fit, predict, evaluate, and guess/slip classes)	One pyBKT implementation is reused through a thin online adapter. Its version and licence are pinned, and its outputs are verified against the library.
Deep tracing	Deep Knowledge Tracing (Piech et al., 2015)	Excluded from the core because it does not provide sufficiently explicit explanations or triggers. It may be considered for later comparison.
Adaptive testing	Computerized Adaptive Testing survey (2024); catsim	Item selection and stopping ideas are adopted. Since IRT ability differs from BKT mastery, catsim is used only if an IRT-based model is selected.

Continued on next page

Table 1 continued from previous page

Area	Existing work	Use in this project
Path planning	Set-to-Sequence concept-aware learning paths (2023)	Concept-aware scoring is used after enforcing hard constraints. Neural ranking is excluded from the core because it cannot guarantee zero prerequisite violations.
Spaced review	Half-Life Regression (Settles and Meeder, 2016); Duolingo implementation	Fixed half-life decay is used as the baseline. Half-Life Regression is considered only if suitable training data are available.
Graph operations	NetworkX	Used for topological ordering, ancestor queries, and cycle detection.
Data	ASSISTments; Junyi Academy	ASSISTments is used for offline sanity checks of the mastery model, while Junyi Academy serves as a reference for designing a realistic prerequisite graph.

3 System Design

This section describes the architecture of AdaptEngine, its major components, how they interact, and the inputs and outputs of each operation. The design follows one rule throughout: the deterministic engine is the authority over validity, mastery, feasibility and state transitions, and everything else works around it.

3.1 Design Principles

The architecture is based on the following quality attributes, which are also used as review criteria for each component.

Table 2: Architecture quality attributes.

#	Attribute	Meaning
Q1	Constraint safety	Prerequisites and daily time budgets are enforced as hard constraints by the engine.
Q2	Determinism	The same inputs and configuration always produce the same decisions and traces.
Q3	Traceability	Every decision includes a machine-readable reason.
Q4	Testability	Modules can be tested independently and produce deterministic results wherever possible.
Q5	Replaceability	Mastery, review, ranking, and planning components can be replaced through clearly defined interfaces.

#	Attribute	Meaning
Q6	Measurable performance	Important operations can be benchmarked using fixed workloads.
Q7	Bounded autonomy	Optional agents may propose or rank options, but cannot bypass the engine's constraints.

3.2 System Architecture

AdaptEngine follows a layered architecture. Each layer has one responsibility and depends only on the layers below it. The lower layers give stable deterministic building blocks, while the higher layers combine them into learning and planning behaviour.

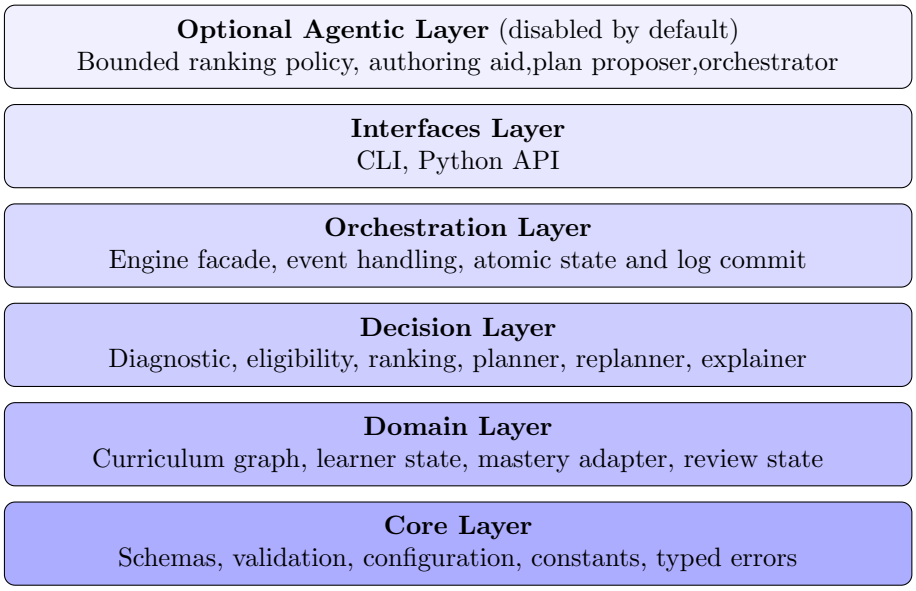


Figure 1: Layered architecture of AdaptEngine. Each layer uses only the layers below it. The agentic layer sits outside the deterministic core.

3.2.1 Core Layer

The Core layer holds the basic structures shared by the whole system: typed schemas, strict validators, configuration values, constants and the error hierarchy. It contains no learning or planning logic. Numerical parameters such as the initial mastery p_0, the threshold θ, the learning transitions T and the difficulty bounds are kept in one immutable configuration whose SHA-256 hash is saved with every run, so a change in a parameter is visible in the experiment record.

3.2.2 Domain Layer

The Domain layer represents the learning problem itself. It contains the curriculum DAG with prerequisite edges, the learner state with mastery and evidence fields, the mastery adapter, and the review-related state such as half-life and last practice day. Curriculum structure is separate from display-only fields such as concept descriptions, which must not change the graph.

3.2.3 Decision Layer

The Decision layer contains the algorithms that decide what the engine should do: diagnostic item selection, activity eligibility and ranking, planning, replanning, review decisions and explanations. These components work on validated domain state. Hard constraints such as prerequisite validity and time budgets are enforced by the engine and are never left to a heuristic or learned ranking.

3.2.4 Orchestration Layer

The Orchestration layer connects domain state and decision components. Its entry point is the Engine facade, which receives typed events and applies the correct state transition. The single mutation interface is

$$(\text{state}', \text{log}') \leftarrow \texttt{Engine.record}(\text{state}, \text{log}, \text{event}).$$

Keeping one controlled boundary makes the workflow consistent and makes later replanning and simulation easier to add. State and log are committed together as one atomic snapshot, so a failure cannot leave them inconsistent.

3.2.5 Interfaces Layer

The Interfaces layer contains a thin command-line interface and a Python API. It only translates external input into typed engine operations and shows the resulting state or errors. No learning or planning logic is placed in the CLI. The same engine can therefore be used from a CLI, a Python program or the simulator without duplicating logic.

3.2.6 Optional Agentic Layer

This layer sits above the deterministic core and is disabled by default, so the system works without network access and provides a deterministic baseline for testing. Four bounded extensions are defined. In each, the agent proposes, reorders or chooses, while the engine validates and decides.

- **A0, Authoring Aid:** an offline assistant that drafts curriculum content such as concept relationships, lesson text and remediation hints. Its output is draft content and must pass schema validation and human review before entering the live curriculum.
- **A1, Plan Proposer:** proposes a day-by-day plan from learner state, goal, budget and deadline. The engine validates every proposal against prerequisite and budget constraints and falls back to the deterministic planner on rejection.
- **A2, Ranking Policy:** re-ranks activities that have already passed the hard-constraint filter, using feedback from past outcomes. It cannot add activities or change eligibility.
- **A3, Orchestrator:** selects the next strategy (continue the plan, remediate, review or re-diagnose) from a closed set of options generated and validated by the engine.

3.3 System Context

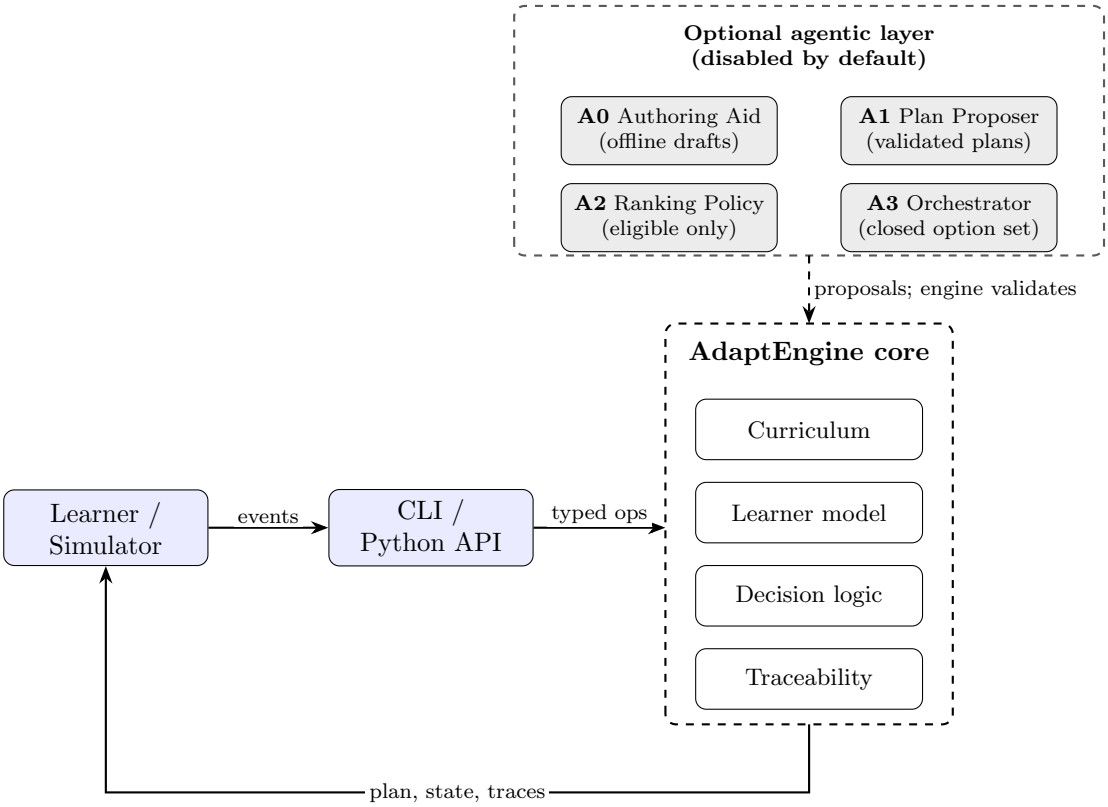


Figure 2: System context. Learners or the simulator interact through a thin CLI or API.

3.4 Major Components

Component	Function	Status
Schemas and validation	Typed dataclasses, strict enums and booleans, finite-probability and range checks, typed errors, configuration hash.	Foundation completed
Curriculum loader	Parses JSON, validates fields, rejects unknown prerequisites and cycles, builds the NetworkX graph.	Partly completed; unknown-prerequisite check ongoing.
Review gate	SHA-256 manifest check, literal `edges_reviewed`, named human sign-off for live content.	Partly completed; enforcement at every live entry point ongoing.
Learner state and mastery (M4)	Immutable state, evidence fields, attempt IDs, Bayes update, adapter to the pinned pyBKT route.	State completed; evidence semantics and adapter pending.
Diagnostic (M2)	Expected-entropy item selection, item cap, cold-start rule, `UNASSESSED` handling.	Pending.
Eligibility and ranking (M3)	Hard filter on prerequisites and budget, score components, deterministic tie-break.	Pending.

Component	Function	Status
Planner and oracle (M3)	Day-by-day packing, feasibility bounds, exact oracle for small instances returning `FEASIBLE`, `INFEASIBLE` or `UNKNOWN`.	Pending.
Review and remediation (M5)	Half-life decay baseline, priority policy under one shared daily budget.	Pending.
Replanner (M6)	Handlers for missed day, failed assessment, deadline change and budget change, preserving completed attempts.	Pending.
Explainer (M7)	Decision-trace records with state, constraint, score components and reason code.	Pending.
Simulator and evaluator (M8)	Transparent synthetic simulator, frozen baselines, paired workloads, metric computation.	Pending.
Engine facade	`Engine.record(state, log, event)` as the single mutation boundary.	Basic version completed.
CLI	Thin wrapper over the API with caller-supplied content paths.	Basic version completed; default review enforcement ongoing.

3.5 Component Interaction and Workflow

The overall workflow is as follows.

1. **Load and validate.** Curriculum and activity files are schema-checked, unknown references and cycles are rejected, and the review manifest is verified. A failure stops here with a typed error and no change to state or log.
2. **Diagnose.** The adaptive diagnostic selects a bounded number of items and produces an initial mastery vector with entropy and `UNASSESSED` flags.
3. **Plan.** The planner filters eligible activities, ranks them, packs each day within the budget, and returns a plan or a feasibility verdict. Every selected and omitted activity gets a decision trace.
4. **Execute and update.** The learner or simulator completes or misses activities. Each outcome is a typed event handled by `Engine.record`, which updates mastery and the log together.
5. **Replan.** A missed day, failed assessment, deadline change or budget change triggers replanning over the remaining horizon, keeping all completed attempts.
6. **Evaluate.** The same engine runs against the simulator with fixed seeds and workloads, and its results are compared with frozen baselines.

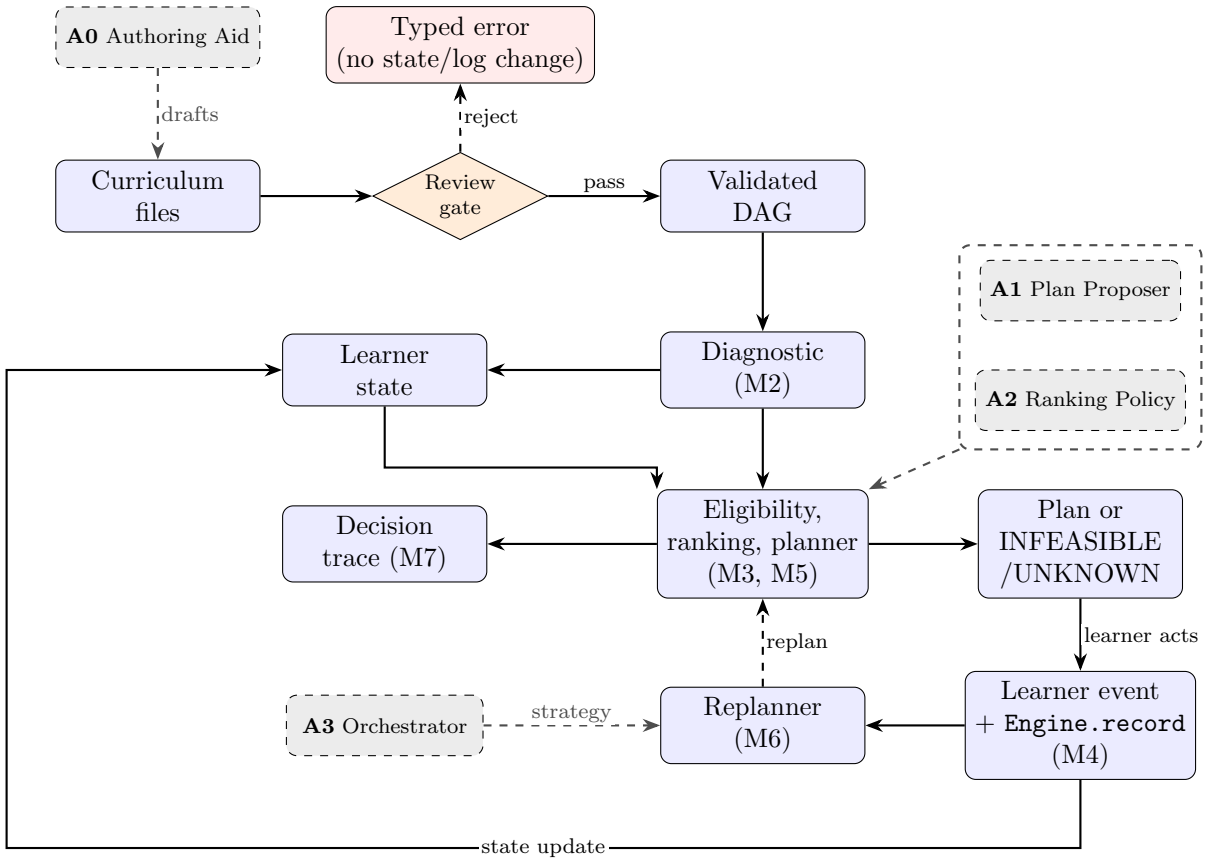


Figure 3: End-to-end workflow

3.6 Inputs and Outputs

Table 4: Expected inputs and outputs of each operation.

Operation	**Inputs**	**Outputs**
Content load	Curriculum JSON, activities JSON, review manifest, and configuration.	Validated DAG and activity table, or a typed error.
Diagnostic	Learner state, goal concepts, and item cap $N_{\max}$.	Mastery p, entropy H, assessed flags, and item log.
Planner	Learner state, goal concepts, deadline in days, and daily budget $B \in \{5, 10, 15, 30\}$ minutes.	Day-by-day schedule, or `INFEASIBLE` with shortfall and corrective options, or `UNKNOWN`; decision trace.
Event recording	Typed event with `attempt_id`, activity, strictly boolean correctness, and day.	New learner state and appended log, or a typed error with state and log unchanged.
Replanning	Event and the current plan.	Revised valid plan preserving completed attempts, or an infeasibility proof.

Operation	Inputs	Outputs
Evaluation	Simulator profile, seed, workload, and baseline name.	Raw run output, configuration hash, code SHA, and metrics.

Boundary rules. The following rules apply at every public entry point.

- Content review is required by default at every live entry point (initialise, show, update and the API). An authoring-only mode exists, is clearly named and prints a warning.
- All validation happens before any updates. A rejected input leaves the learner and log files unchanged.
- Event kinds are strict enumerations, correctness must be a real boolean (the string `"false"` is rejected), and probabilities must be finite and in range.
- Unknown prerequisite references are rejected before graph edges are created.

3.7 Data Model

- **Concept:** `id`, `description`, `prerequisites` (existing concept IDs only).
- **Activity:** `id`, `type` (`LESSON` or `EXERCISE`), `target_concept` (exactly one primary concept), `duration` (integer minutes, indivisible), `difficulty` in $[0.05, 0.95]$ for exercises.
- **LearnerConcept:** mastery p, lessons completed, observed responses, assessed flag, half-life in days, last practice day, and the set of seen `attempt_id`s.
- **Event:** strict kind (completed, missed, failed assessment, deadline changed, budget changed), `attempt_id`, and a payload validated per kind.
- **Plan:** ordered mapping from day to activities, plus a feasibility status and a decision-trace log.

4 Technical Feasibility

This section lists the tools, datasets and infrastructure needed, explains why the design is techincally feasible, describes how the system will be evaluated, and states the main challenges, assumptions and limitations.

4.1 Tools, Frameworks, Datasets and Infrastructure

Table 5: Tools and frameworks.

Need	Choice	Reason
Language	Python 3	Fast to develop, rich libraries, easy to test.

Need	Choice	Reason
Graph operations	NetworkX	Supports topological ordering, ancestor queries, and cycle detection, avoiding the need for custom graph code.
Mastery model	pyBKT behind a thin adapter; custom update kept as a reference oracle	Reuses an established model-fitting and prediction implementation instead of building a general framework from scratch.
Numerics	NumPy and SciPy only where library results are needed	Keeps dependencies small.
Testing	pytest, coverage	Supports reference-value, boundary, and regression tests.
Code quality	Ruff for linting and formatting, CI on every push	Keeps the codebase consistent and helps detect problems early.
Interfaces	Python API and a thin CLI; Streamlit only if time permits	Keeps the engine independent of the user interface.
Reproducibility	SHA-256 configuration hash, fixed seeds, saved code SHA, and raw outputs	Allows each reported run to be reproduced and checked.

Datasets and simulator. Public datasets do not allow interactive tests of daily time limits, missed days or changed deadlines, so a hybrid strategy is used.

- **Transparent synthetic simulator (primary).** It provides every field the planner needs: durations, prerequisites and ground-truth mastery. It supports strict budgets and skipped days, and it is used for the full M1–M8 evaluation loop.
- **ASSISTments (offline check).** Real student response logs with skill tags are used only to sanity-check the mastery model, for example by measuring next-answer prediction. It has no prerequisite graph or lesson durations, so it is not used interactively.
- **Junyi Academy (design reference).** It has a real prerequisite structure and is used to shape a realistic prerequisite DAG. It lacks durations and review labels, so it is not a data source for the planner.

Missing fields are handled openly. If no usable timing data exists, placeholder durations are used and marked as such in the code. Skills that do not map cleanly between datasets are left out and no prerequisite links are invented. Concepts with too little history receive a default median half-life.

Infrastructure. The whole system runs on a laptop CPU. No GPU, no server and no network access are needed at runtime, because the engine is deterministic and the optional agent layer is disabled by default. But the agentic layer when used may need GPU access or internet access.

4.2 Technical Feasibility of the Design

- **A working base already exists.** The Week 2 foundation has a four-concept curriculum with 20 activities, a NetworkX graph, immutable learner state, a response-driven mastery

update, typed events, a CLI and a review manifest. It passes 47 tests with 84.95% statement coverage, and repeated smoke runs give identical output. The remaining work builds on this base.

- **The scope is reduced to a manageable size.** Custom A* search was removed, one mastery model is used, and the review model starts as a simple fixed half-life baseline. Heavy generic work is handed to existing libraries.
- **The planner problem is small.** With a handful of concepts and indivisible durations, a deterministic scheduler and an exact oracle for small instances are cheap to run and easy to verify.
- **Staged delivery with testable outputs.** Each stage ends with a result that can be run and checked by acceptance tests before the next stage begins: boundary repair first, then the model adapter and static plan, then the diagnostic and personalized planner, then review and replanning, and finally traces and evaluation. Every stage builds only on components that have already passed their tests, which keeps faults easy to locate and avoids integrating several unverified components at once.
- **Measurement is built in.** Every module has a frozen baseline and a binary acceptance test, so progress can be shown with evidence.

4.3 Evaluation Protocol

Table 6: Evaluation design.

Element	Design
Simulator	One transparent simulator is used, with learning and forgetting dynamics that differ from the planner's BKT-style model. Parameters are sampled from published ranges, their sources are recorded, and at least one mismatched variant is included.
Frozen baselines	Fixed-order and random diagnostic selection with the same item budget; a static plan based on prerequisite order; and a greedy plan that selects the next eligible activity without review scoring.
Workloads	All methods use the same learner profiles, goals, deadlines, budgets, and random seeds.
Splits	Synthetic profiles are divided into tuning and held-out sets. Parameters are tuned only on the tuning set and never on the held-out set.
Repeatability	Running the same configuration twice with identical random seeds must produce identical raw results.

The metrics are grouped as follows.

Table 7: Evaluation metrics.

Group	Metrics
Constraint adherence	Prerequisite-violation count, daily-budget violation count, and plan feasibility rate (valid plan or correct infeasibility report).
Diagnostic and predictive accuracy	Mean absolute error between estimated and ground-truth mastery at equal item count; Brier score of P_c on held-out attempts; and AUC of next-attempt correctness, including the offline ASSISTments check.
Learning outcomes	Simulated goal mastery by the deadline for each profile; review retention for reviewed versus non-reviewed concepts.
Efficiency and explainability	Replanning runtime in milliseconds; explanation coverage, measured as the proportion of decisions with a complete decision trace.

4.4 Potential Implementation Challenges, Assumptions and Limitations

Assumptions:

- The prerequisite graph is an accurate DAG and its edges are hard constraints.
- Every activity has a fixed, indivisible duration, and partial completion is not modelled. A missed day means zero minutes of activity.
- Each exercise maps to exactly one primary concept, so every answer updates one concept without ambiguity.
- Difficulty is a continuous value between 0.05 and 0.95.
- Responses are strictly binary (correct or incorrect), and this signal is enough to update the belief about mastery.
- Mastery is a hidden state that can only be estimated from responses.

Challenges and Limitations:

Table 8: Implementation challenges, risks and mitigations.

Challenge or limitation	Risk	Mitigation
Heuristic parameters	p_0, θ, T, $g(d)$ and $s(d)$ are not fitted to data.	State them as synthetic; run a small sensitivity analysis; make no fitted-performance claim.
pyBKT mapping	The continuous $g(d)$ and $s(d)$ do not exist in pyBKT, which uses item classes.	Map difficulty to classes explicitly, report every mapped parameter, and test against library output.

Continued on next page

Table 8 continued from previous page

Challenge or limitation	Risk	Mitigation
Simulator bias	A simulator built on the planner's own rules would inflate results.	Use a different dynamics family, mismatched variants, and held-out profiles.
Entropy interpretation	Entropy can rise after a surprising answer and does not reflect evidence count.	Use expected entropy reduction with a hard item cap, and judge quality by held-out MAE and Brier score.
Review model	Language-learning half-life features may not transfer to other type of concepts.	Start with a fixed half-life baseline and label it clearly.
Binary responses	Partial credit and hints are ignored.	State this as an assumption; single-concept tagging keeps the model unambiguous.
Indivisible activities	Real learners may pause a lesson.	Stated this as an assumption and a limitation of the prototype.
Persistence	Writing state and log as two files can leave them inconsistent after a failure.	Use one atomic snapshot with an injected write-failure test.

5 Implementation Plan

This section breaks the proposed solution into implementation tasks, lists the core functionalities, gives the expected outcomes, and states the next steps.

5.1 Current Status

Table 9: Current status

Status	Work	Evidence
Completed	Week 0–2 foundation: problem boundary, literature and data decisions, layered architecture, four-concept curriculum with 20 activities, NetworkX prerequisite graph, immutable learner state, BKT-style response update, typed events, `Engine.record`, thin CLI, review manifest, and configuration hash.	47 passing tests, 84.95% statement coverage, successful Ruff lint and format checks, and two identical deterministic smoke runs.

Continued on next page

Table 9 continued from previous page

Status	Work	Evidence
Ongoing	Repairs to the Week 2 foundation following the mentor review and together with completion of the adaptive diagnostic (M2).	Ongoing
Pending	Personalized planner (M3), difficulty policy (M4), review and remediation (M5), replanning (M6), decision traces (M7), and comparative evaluation (M8).	Target weeks W4–W8.

5.2 Task Breakdown

Week	Goal	Tasks	Evidence
W3	Complete the adaptive diagnostic (M2), integrate the model adapter and produce the first static plan.	(1) Adaptive diagnostic: expected-entropy item selection, cold-start rule, `UNASSESSED` handling, item cap and stopping rule. (2) Small set of synthetic learners with known mastery on the four-node fixture; comparison with fixed and random selection at equal item budget. (3) Pinned pyBKT adapter with a parity test against the library. (4) Learner-state refinement to keep instruction and assessment evidence separate. (5) Input validation and content-review checks applied consistently across the API and CLI. (6) Tiny static prerequisite-valid plan on the four-node fixture.	TC1.2, TC2.1, TC2.2 pass; diagnostic never exceeds $N_{\max}$; one complete run command with saved code SHA, environment and raw output.
W4	Personalized planner and difficulty policy.	(1) Eligibility filter and deterministic ranking driven by the diagnostic output. (2) Day-by-day packing for 5, 10, 15 and 30 minute budgets. (3) Next-item difficulty policy. (4) Feasibility bounds, exact oracle, `INFEASIBLE` and `UNKNOWN`. (5) Larger curriculum fixture.	TC2.3, TC3.1–3.6, TC4.2 pass; packing counterexample and timeout fixture work.

Week	Goal	Tasks	Evidence
W5	Review, remediation and replanning.	(1) Fixed half-life review baseline and urgency score. (2) Priority policy: remediation, then review, then new lessons, under one shared budget. (3) Remediation insertion when $p < 0.40$. (4) Handlers for missed day, failed assessment, deadline change and budget change. (5) Preserve completed attempts; add deadline and budget payload contracts.	TC5.1–5.3, TC6.1–6.4 pass.
W6	Traces and comparative harness.	(1) Decision-trace records for inclusion, omission, review and difficulty change. (2) Full transparent simulator with provenance of every parameter. (3) Frozen baselines: fixed and random diagnostic, static plan, greedy plan. (4) Paired workloads, seeds and tuning/held-out split. (5) Metric computation.	TC7.1, TC8.1 pass; raw results saved and repeatable.
W7	Stress, mismatch and one measured improvement.	(1) Mismatched simulator variants and stress cases. (2) Sensitivity run over the heuristic parameters. (3) One measured improvement, for example frontier replanning versus full recomputation with identical decisions. (4) Optional bounded ranking policy, only if all core tests pass.	TC8.2; comparison tables; limitations section.
W8	Final demonstration and report.	(1) Clean, reproducible CLI and API demonstration for several synthetic learners. (2) Final evaluation report. (3) Documentation clean-up (4) Preparation for explanation and modification review.	Tagged repository, final report, reproducible run command.

5.3 Core functionalities to be implemented by the end

1. **Boundary-safe curriculum and learner state (M1).** Mandatory review gate, strict schema and type validation, unknown-reference and cycle rejection, unchanged state and log on rejection.

2. **Online mastery update and difficulty policy (M4).** A pinned established-model

adapter verified against the library, the reference Bayes update as an oracle, idempotent retry handling, and a next-item rule that targets a predicted success probability.

3. **Bounded adaptive diagnostic (M2).** Expected-entropy item selection, cold-start rule, item cap and stopping rule, compared with fixed and random selection at equal item budget.

4. **Prerequisite-valid, time-budgeted planner (M3).** Hard eligibility filter, deterministic ranking, daily packing, feasibility bounds, exact oracle with `FEASIBLE`, `INFEASIBLE` and `UNKNOWN`.

5. **Review and remediation (M5).** Forgetting-risk scoring and a fixed priority order under one shared daily budget.

6. **Dynamic replanning (M6).** Four event types, revised plans over the remaining horizon, completed attempts preserved.

7. **Explanations (M7).** A machine-readable trace for every inclusion, omission, review and difficulty change.

8. **Comparative evaluation (M8).** Simulator, frozen baselines, paired workloads and held-out metrics.

5.4 Acceptance Tests and Traceability

The authoritative record is the register of 23 core test cases below. Each ID states its milestone and sequence number, for example TC4.2 is the second test case of M4.

ID	Scenario	Expected result	Week
M1 – Curriculum and learner state			
TC1.1	Load the prerequisite graph and inject a cycle.	Valid graph confirmed acyclic; cycle rejected at load time with a typed error naming the concepts.	W2
TC1.2	Load valid and invalid curriculum, learner-state, event and configuration fixtures, and content that has not passed review.	Valid fixtures load with correct types; invalid or unreviewed inputs are rejected with typed errors before any change to state or log.	W2–3
M2 – Adaptive diagnostic			
TC2.1	Run the diagnostic on the four-node fixture with synthetic learners of known mastery.	Stops at the uncertainty threshold or the item cap, never above $N_{\max}$; estimation error is no worse than fixed and random selection at equal item budget.	W3
TC2.2	Diagnose concepts with no previous observations.	Prior state preserved, concept marked `UNASSESSED` and treated as unmastered.	W3
TC2.3	Two learners with different prior mastery, same goal and budget.	Different, individually valid plans.	W4
M3 – Planning and feasibility			

ID	Scenario	Expected result	Week
TC3.1	Plan on a graph with an unmet mandatory prerequisite.	No lesson for the locked concept before its prerequisite is evidence-mastered.	W4
TC3.2	Plan under a fixed daily budget.	No day exceeds the budget.	W4
TC3.3	Same learner and goal, 5 versus 30 minute budget.	Materially different pacing or number of days.	W4
TC3.4	Goal cannot fit before the deadline.	`INFEASIBLE` with shortfall and corrective options.	W4
TC3.5	The $8 + 8 + 4$ minute packing fixture with a 10 minute budget.	Bounds pass but the oracle reports `INFEASIBLE` (three days needed).	W4
TC3.6	Force the feasibility search past its time limit.	Oracle returns `UNKNOWN`, never a false `INFEASIBLE`.	W4
M4 – Online mastery update and difficulty			
TC4.1	Incorrect response on a low-mastery concept; adapter versus library on the same response sequence.	For $p = 0.5$, $d = 0.5$: 0.1707 before and 0.2122 after the learning transition; adapter output matches the library.	W2–3
TC4.2	Select the next exercise for a given mastery.	Exercise whose predicted success is nearest the target; ties by the stated rule.	W3–4
M5 – Review and remediation			
TC5.1	Concept already evidence-mastered above θ.	Omitted from new learning, trace cites the evidence.	W5
TC5.2	Failed exercise leaves $p < 0.40$.	Remediation lesson inserted; review state and flag updated after the specified cycles.	W5
TC5.3	High review urgency with spare capacity.	Review takes priority over lower-urgency new content.	W5
M6 – Replanning			
TC6.1	Learner misses a scheduled day.	New valid plan; completed work not lost or duplicated.	W5
TC6.2	Failed-assessment event.	State updated and a valid revised plan with remediation.	W5
TC6.3	Deadline changed after planning.	Valid revised plan or proof of `INFEASIBLE`.	W5
TC6.4	Daily budget changed.	Valid revised plan that satisfies the new budget.	W5
M7 – Decision traces			
TC7.1	Plan with selected and omitted activities.	Every decision traceable to state, constraints and score components.	W5–6
M8 – Evaluation			
TC8.1	Run the harness twice with identical seeds and configuration.	Identical raw results; frozen baselines unchanged.	W6

ID	Scenario	Expected result	Week
TC8.2	Frontier replanning versus full recomputation on the same events.	Identical decisions; latency recorded for both.	W6–7

5.5 Expected Outcomes of the Implementation Phase

- A boundary-safe, reproducible engine in which unreviewed or malformed input is rejected before any change to state or log.
- A bounded adaptive diagnostic that gives a starting mastery estimate, compared with fixed and random selection at equal item budget.
- A prerequisite-valid, budgeted planner that gives different valid plans to several synthetic learners with different skills, deadlines and budgets, and reports infeasibility clearly.
- Online mastery updates with an explicit difficulty policy, review and remediation under one shared budget, and valid replanning for all four event types.
- A decision trace for every inclusion, omission, review and difficulty change.
- An evaluation report with frozen baselines, paired workloads and held-out metrics, showing at least one measured improvement and stating its limits.

5.6 Planned Next Steps

1. **Complete the adaptive diagnostic (M2).** Implement expected-entropy item selection, the cold-start rule, `UNASSESSED` handling, the item cap and the stopping rule, and compare it with fixed and random selection at equal item budget on synthetic learners with known mastery.
2. **Integrate the model adapter.** Add the pinned pyBKT adapter with a parity test against the library, and keep the reference update as an independent oracle.
3. **Strengthen the foundation.** Keep instruction exposure and observed assessment evidence separate in learner state, and apply input validation and content-review checks consistently across the API and CLI.
4. **Produce the first static plan.** Generate a tiny prerequisite-valid plan on the four-node fixture and save the first reproducible run with code SHA, environment and raw output.
5. **Begin the personalized planner (M3).** Start the eligibility filter and ranking on top of the diagnostic output, ready for Week 4.

6 Conclusion

AdaptEngine addresses the limits of fixed course sequences by estimating what each learner knows, planning what to study within a goal, deadline and daily time budget, and adjusting as the learner progresses. Its core principle is the separation of validity from preference: prerequisites and time budgets are hard constraints enforced by a deterministic engine, while a single BKT-style mastery model and explicit scores only rank the activities that pass these constraints. Because every inclusion, omission, review and difficulty change is recorded in a

machine-readable trace, each decision can be explained and verified. The system is built in stages from diagnosis through planning, review and replanning, and is evaluated against frozen baselines on paired workloads. The results will be reported as prototype-scale, with the heuristic parameters, small curriculum and binary responses stated as limitations.